AI Foundations: VS Code + R Laptop Setup Guide

These instructions cover macOS and Windows. Platform-specific steps are noted where they differ.

1. Download workshop files

- Go to the workshop repository on GitHub (link provided by your instructor)
- Click the green **Code** button, then click **Download ZIP**
- Extract the ZIP file to a location you'll remember (e.g., your Desktop or Documents folder)

> You do not need Git installed. The ZIP download includes everything you need.

2. Install Homebrew (macOS only)

If you already have Homebrew, skip this step. Otherwise, open **Terminal** (`Cmd + Space` , type `Terminal` , press `Enter`) and run:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Follow the prompts. When it finishes, **close and reopen Terminal** so the `brew` command is available.

3. Install R (macOS)

Option A: Homebrew (if you completed Step 2)

```
brew install r
```

Option B: CRAN installer (recommended for most users)

- Download the latest `.pkg` from https://cran.r-project.org/bin/macosx/
- Open the installer and follow the prompts

3. Install R (Windows)

- Download the latest `.exe` from https://cran.r-project.org/bin/windows/base/
- Run the installer and follow the prompts

4. Request an API key

Email Nicholas Warren at DISSC (dissc@yale.edu) to request an AWS Bedrock API key for Claude. **Request this before the workshop;** you'll need the key in Step 5. This key will expire after the workshop. For long-term access, contact dissc@yale.edu.

5. Configure AWS Bedrock credentials (macOS)

In Terminal, run `nano ~/.zprofile` and add these three lines, replacing `TOKEN_HERE` with your API key from Step 4:

```
export AWS_REGION="us-east-1"
export AWS_BEARER_TOKEN_BEDROCK="TOKEN_HERE"
export CLAUDE_CODE_USE_BEDROCK=1
```

Save with `Ctrl + O`, press `Enter`. Exit with `Ctrl + X`.

5. Configure AWS Bedrock credentials (macOS, continued)

Back in Terminal, load the new settings into your current session:

```
source ~/.zprofile
```

> If your Mac uses `bash` instead of `zsh` (check with `echo $SHELL`), use `~/.bash_profile` instead.

> **Treat your API key like a password.** Do not share it or commit it to version control.

5. Configure AWS Bedrock credentials (Windows)

Click the Start menu, type `environment variables`, and select **Edit environment variables for your account**. Under **User variables**, click **New** to add each:

Variable	Value
`AWS_REGION`	`us-east-1`
`AWS_BEARER_TOKEN_BEDROCK`	your API key
`CLAUDE_CODE_USE_BEDROCK`	`1`

After saving, restart any open terminals or applications.

6a. Install Claude Code

Follow the **Terminal** install instructions for your platform at https://code.claude.com/docs/en/overview

Open a **new** terminal window so it picks up the credentials from Step 5, then verify the installation by running `claude --version`.

6b. Configure Claude Code safety settings (macOS)

Claude Code can run commands on your computer. These settings add safety guardrails that reduce the risk of accidental file deletion, data loss, or changes outside your project.

- Open **Finder** and press `Cmd + Shift + G` (Go to Folder)
- Type `~/.claude` and press Enter
  - If the folder doesn't exist: press `Cmd + Shift + G` again, type `~/`, press Enter, then create a new folder named `.claude` (`File > New Folder`)
- From the workshop files you downloaded in Step 1, open `files > machine-setup > claude`
- Copy `settings.json` into the `~/.claude` folder

> Read `settings-README.md` in the same folder to learn what each setting does and how to customize them.

6b. Configure Claude Code safety settings (Windows)

Claude Code can run commands on your computer. These settings add safety guardrails that reduce the risk of accidental file deletion, data loss, or changes outside your project.

- Open **File Explorer** and type `%USERPROFILE%\.claude` in the address bar, then press Enter
  - If the folder doesn't exist: navigate to `%USERPROFILE%`, then create a new folder named `.claude` (`Right-click > New > Folder`)
- From the workshop files you downloaded in Step 1, open `files > machine-setup > claude`
- Copy `settings.json` into the `.claude` folder

> Read `settings-README.md` in the same folder to learn what each setting does and how to customize them.

7. Install Quarto (macOS)

```
brew install --cask quarto
```

If you don't have Homebrew, download the installer from https://quarto.org/docs/get-started/

7. Install Quarto (Windows)

- Download the installer from https://quarto.org/docs/get-started/
- Run the installer and follow the prompts

8. Install VS Code

Visual Studio Code (VS Code) is a free, open-source code editor made by Microsoft. It supports R, Python, and many other languages through extensions. We use it in this workshop as your main working environment.

- Download and install from https://code.visualstudio.com/

9. Install VS Code extensions

- Open VS Code
- Click the **Extensions** icon in the left sidebar (`Cmd + Shift + X` / `Ctrl + Shift + X`)
- Search for and install each of these:
  - **R** by REditorSupport
  - **Quarto** by Quarto

10. Configure VS Code

> If VS Code was already open during Step 5, quit and reopen it so it picks up the new credentials.

- `File > Open Folder` and select the `vs-code-and-r` folder from Step 1
- When prompted, click `Yes, I trust the authors`

10. Configure VS Code (continued)

- Open the Command Palette (`Cmd + Shift + P` / `Ctrl + Shift + P`)
- Type `User Settings JSON` and select `Preferences: Open User Settings (JSON)`
- If the file is empty or contains only `{}` , replace everything with the contents of `files/machine-setup/vscode/settings.json` and save
- If you already have settings, add the keys from `settings.json` into your existing JSON

11. Start an R session

- In VS Code's left sidebar, click the top icon (two overlapping pages) to open the **Explorer**, which shows your project files
- Navigate to `files > code` and click `example.qmd` to open it
- Click `R: (not attached)` in the bottom status bar, or run any code chunk — this will start the R session
- If prompted, select the R installation from Step 3
- An **R Terminal** will appear in the bottom panel with an `>` prompt

12. Install R packages

In the R Terminal, run:

```
install.packages(c("tidyverse", "languageserver"))
```

- `tidyverse`: the workshop's dependencies (also installs `broom` and `scales`)
- `languageserver`: powers code completion, go-to-definition, and diagnostics in VS Code

13. Verify your setup

R: In the R Terminal, run `library(tidyverse)`. You should see a startup message listing attached packages.

Claude Code: Open a terminal in VS Code (`Ctrl + ` `) and run:

```
claude "What is 2 + 2?"
```

You should see Claude respond with an answer.